

Satellite Constellation System

Реализация паттернов Transactional Outbox и Inbox
Конструирование программного обеспечения

Сделано для МГТУ им. Баумана х БЮРО 1440
Семинар 12

29 мая 2026 г.

Содержание

1	Описание проекта	2
1.1	Исходная проблема	2
1.2	Решение	2
2	Архитектура и паттерны	2
2.1	Концептуальная схема	2
2.2	Используемые паттерны	3
3	Структура проекта	3
4	Реализация Transactional Outbox	4
4.1	Таблица outbox (PostgreSQL)	4
4.2	Сохранение события в той же транзакции	4
4.3	Планировщик отправки	4
5	Реализация Inbox (идемпотентность)	5
5.1	Таблица inbox	5
5.2	Consumer с проверкой дубликатов	5
6	Запуск и тестирование	6
6.1	Предварительные требования	6
6.2	Файл docker-compose.yml	6
6.3	Запуск сервисов	6
6.4	Проверка	7
7	Диаграмма потока событий	7
8	Заключение	8

1 Описание проекта

Система управления спутниками состоит из нескольких микросервисов, которые обмениваются асинхронными сообщениями через **Apache Kafka**. Основной сервис (**space-operati**) управляет спутниками (создание/удаление) и должен надёжно уведомлять **telemetry-service** об этих изменениях.

1.1 Исходная проблема

- Прямая отправка события в Kafka после записи в БД не гарантирует атомарности → при сбое между записью и отправкой данные теряются.
- Kafka доставляет сообщения **at-least-once**, что приводит к дубликатам на стороне потребителя.

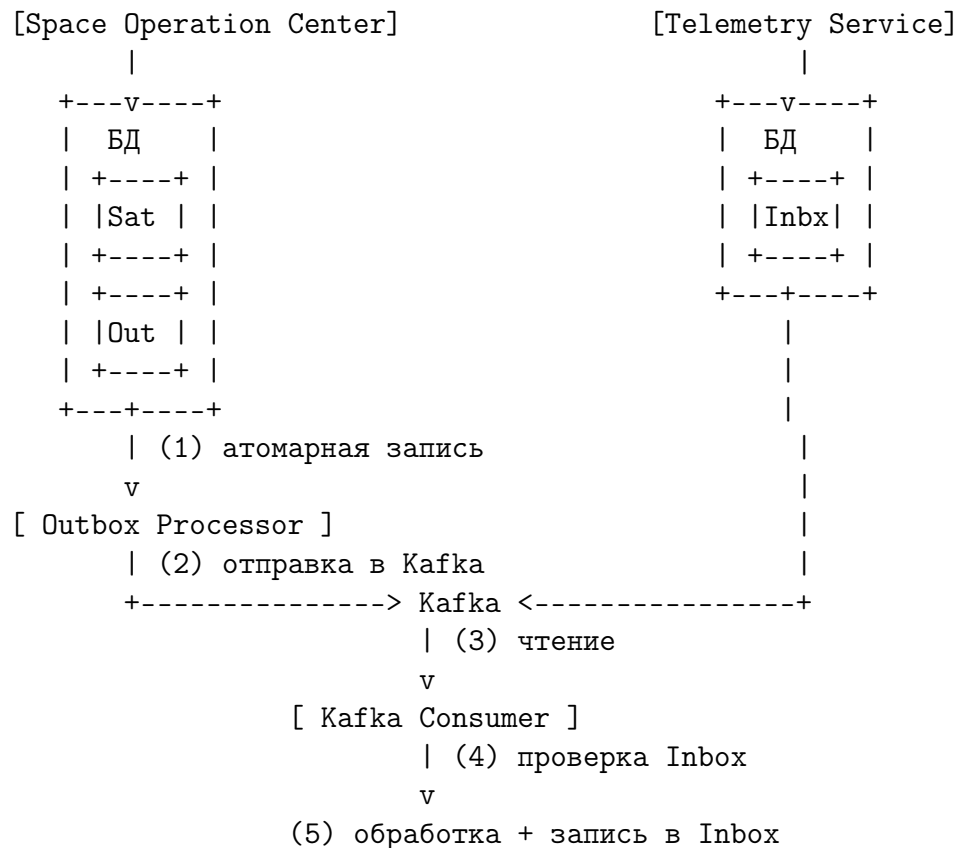
1.2 Решение

Внедрены два паттерна отказоустойчивости:

1. **Transactional Outbox** (на стороне отправителя) – события сначала сохраняются в БД в одной транзакции с бизнес-операцией, затем фоновый процесс отправляет их в Kafka.
2. **Inbox + идемпотентность** (на стороне получателя) – уникальный **event_id** исключает повторную обработку дубликатов.

2 Архитектура и паттерны

2.1 Концептуальная схема



2.2 Используемые паттерны

Паттерн	Где применяется	Назначение
Transactional Outbox	space-operation-center	Гарантирует, что событие будет отправлено в Kafka тогда и только тогда, когда бизнес-операция успешно зафиксирована в БД.
Outbox Scheduler	space-operation-center	Фоновый процесс, читающий таблицу <code>outbox</code> и отправляющий события в Kafka. При успехе пометает запись как <code>SENT</code> .
Inbox	telemetry-service	Таблица <code>inbox</code> хранит ID уже обработанных событий для обеспечения идиоматичности.
Idempotent Consumer	telemetry-service	Перед обработкой события проверяется наличие <code>event_id</code> в <code>inbox</code> – если есть, событие игнорируется.

3 Структура проекта

Ниже представлено полное дерево каталогов. – новые/изменённые файлы (относительно базовой реализации Kafka), остальные – неизменная часть.

```
satellite-constellation-system/  
+-- build.gradle.kts  
+-- settings.gradle.kts  
+-- docker-compose.yml  
+-- space-operation-center/  
|   +-- build.gradle.kts  
|   +-- src/main/  
|       +-- java/com/example/spacecenter/  
|           |   +-- SpaceOperationCenterApplication.java  
|           |   +-- service/  
|           |       |   +-- SatelliteService.java  
|           |       |   +-- OutboxScheduler.java  
|           |   +-- domain/outbox/Outbox.java  
|           |   +-- dto/SatelliteEvent.java  
|           |   +-- repository/OutboxRepository.java  
|       +-- resources/  
|           +-- application.yml  
|           +-- schema.sql  
+-- telemetry-service/  
|   +-- build.gradle.kts  
|   +-- src/main/  
|       +-- java/com/example/telemetry/  
|           |   +-- dto/SatelliteEvent.java
```

```
|      |  +-- domain/inbox/Inbox.java
|      |  +-- repository/InboxRepository.java
|      |  +-- listener/SatelliteEventListener.java
|      |  +-- service/SatelliteStorageService.java
|      +-- resources/
|          +-- application.yml
|          +-- schema.sql
+-- mission-scheduler/                                (без изменений)
```

4 Реализация Transactional Outbox

4.1 Таблица outbox (PostgreSQL)

Листинг 1: DDL для outbox

```
1 CREATE TABLE outbox (
2     id BIGSERIAL PRIMARY KEY,
3     aggregate_id VARCHAR(255) NOT NULL,
4     event_type VARCHAR(50) NOT NULL,
5     payload JSONB NOT NULL,
6     created_at TIMESTAMP NOT NULL DEFAULT NOW(),
7     status VARCHAR(20) NOT NULL DEFAULT 'PENDING'
8 );
9 CREATE INDEX idx_outbox_status ON outbox(status);
```

4.2 Сохранение события в той же транзакции

Листинг 2: SatelliteService.java (фрагмент)

```
1 @Transactional
2 public Satellite createSatellite(Satellite satellite) {
3     Satellite saved = satelliteRepository.save(satellite);
4     createOutboxEvent(saved.getId().toString(), "CREATED", saved);
5     return saved;
6 }
7
8 private void createOutboxEvent(String aggregateId, String eventType,
9     Object payload) {
10     String eventId = UUID.randomUUID().toString();
11     SatelliteEvent event = new SatelliteEvent(eventId, aggregateId,
12         eventType, payload);
13     String json = objectMapper.writeValueAsString(event);
14     outboxRepository.save(new Outbox(aggregateId, eventType, json));
15 }
```

4.3 Планировщик отправки

Листинг 3: OutboxScheduler.java

```
1 @Scheduled(fixedDelayString = "${outbox.scheduler.fixed-delay:5000}")
2 @Transactional
```

```

3 public void processOutbox() {
4     List<Outbox> pending = outboxRepository.findByStatus(PENDING);
5     for (Outbox record : pending) {
6         try {
7             kafkaTemplate.send("satellite-events",
8                               record.getAggregateId(), record.getPayload());
9             outboxRepository.updateStatus(record.getId(), SENT);
10        } catch (Exception e) {
11            log.error("Failed to send, will retry later", e);
12        }
13    }
14 }

```

5 Реализация Inbox (идемпотентность)

5.1 Таблица inbox

Листинг 4: DDL для inbox

```

1 CREATE TABLE inbox (
2     event_id VARCHAR(255) PRIMARY KEY,
3     aggregate_id VARCHAR(255) NOT NULL,
4     event_type VARCHAR(50) NOT NULL,
5     processed_at TIMESTAMP NOT NULL DEFAULT NOW()
6 );
7 CREATE INDEX idx_inbox_aggregate ON inbox(aggregate_id);

```

5.2 Consumer с проверкой дубликатов

Листинг 5: SatelliteEventListener.java

```

1 @KafkaListener(topics = "satellite-events", groupId =
2     "telemetry-group")
3 @Transactional
4 public void handleSatelliteEvent(SatelliteEvent event) {
5     if (inboxRepository.existsById(event.getEventId())) {
6         log.info("Duplicate event {} ignored", event.getEventId());
7         return;
8     }
9     inboxRepository.save(new Inbox(event.getEventId(),
10     event.getAggregateId(), event.getEventType()));
11
12     // -
13     if ("CREATED".equals(event.getEventType())) {
14         satelliteStorageService.addSatellite(event.getAggregateId());
15     } else if ("DELETED".equals(event.getEventType())) {
16         satelliteStorageService.removeSatellite(event.getAggregateId());
17     }
18 }

```

Важно: `enable-auto-commit: false` и `@Transactional` гарантируют атомарность записи inbox и обработки.

6 Запуск и тестирование

6.1 Предварительные требования

- Docker & Docker Compose
- Java 17
- Gradle (или использование встроенного wrapper)

6.2 Файл docker-compose.yml

Листинг 6: docker-compose.yml

```
1 version: '3'
2 services:
3   postgres-space:
4     image: postgres:15
5     environment:
6       POSTGRES_DB: space_center
7       POSTGRES_USER: postgres
8       POSTGRES_PASSWORD: password
9     ports:
10      - "5432:5432"
11   postgres-telemetry:
12     image: postgres:15
13     environment:
14       POSTGRES_DB: telemetry
15       POSTGRES_USER: postgres
16       POSTGRES_PASSWORD: password
17     ports:
18      - "5433:5432"
19   zookeeper:
20     image: confluentinc/cp-zookeeper:latest
21     environment:
22       ZOOKEEPER_CLIENT_PORT: 2181
23     ports:
24      - "2181:2181"
25   kafka:
26     image: confluentinc/cp-kafka:latest
27     depends_on:
28       - zookeeper
29     environment:
30       KAFKA_BROKER_ID: 1
31       KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
32       KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
33       KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
34     ports:
35      - "9092:9092"
```

6.3 Запуск сервисов

Терминал 1

```
cd space-operation-center
./gradlew bootRun
```

```
# Терминал 2
cd telemetry-service
./gradlew bootRun
```

6.4 Проверка

Создайте спутник через REST API space-operation-center:

```
curl -X POST http://localhost:8080/satellites \
  -H "Content-Type: application/json" \
  -d '{"name":"Sputnik-1","state":"ACTIVE"}'
```

Через несколько секунд в логах telemetry-service появится:

```
Processed event <uuid> for satellite 1
Satellite 1 added to telemetry storage
```

При ручном повторе отправки (например, перезапуск планировщика) Kafka может доставить дубликат, но inbox отфильтрует его:

```
Duplicate event <uuid> ignored
```

7 Диаграмма потока событий

```
Client -> SOC: POST /satellites
SOC -> DB: BEGIN TX
SOC -> DB: INSERT satellite
SOC -> DB: INSERT outbox (PENDING)
SOC -> DB: COMMIT TX
SOC --> Client: 200 OK

loop Every 5 sec
  Scheduler -> DB: SELECT pending outbox
  Scheduler -> Kafka: send(event)
  Scheduler -> DB: UPDATE status -> SENT
end

Kafka -> Telemetry: deliver event
Telemetry -> InboxDB: SELECT exists(event_id)
alt event_id not found
  Telemetry -> InboxDB: INSERT into inbox
  Telemetry -> Telemetry: process business logic
  Telemetry -> Kafka: commit offset
else event_id already exists
  Telemetry -> Telemetry: ignore duplicate
end
```

Рис. 1: Sequence diagram в текстовом виде

8 Заключение

Реализованные паттерны **Transactional Outbox** и **Inbox** обеспечивают:

- **Гарантированную доставку** – событие не будет потеряно даже при сбое сразу после записи в БД.
- **Идемпотентность** – повторные доставки (at-least-once) не приводят к двойной обработке.
- **Согласованность** – между `space-operation-center` и `telemetry-service` сохраняется актуальное состояние спутников.

Такой подход рекомендован для production-систем с асинхронным взаимодействием и высокими требованиями к надёжности.

Дополнительные материалы

- [Pattern: Transactional Outbox](#)
- [Idempotent Consumer](#)
- [Spring Kafka Reference](#)